

Topic 4

**High-Level
Design, Low-
Level Design
and Database
Design**

Learning Outcomes

By the end of this lesson, students should be able to:

High-Level Design

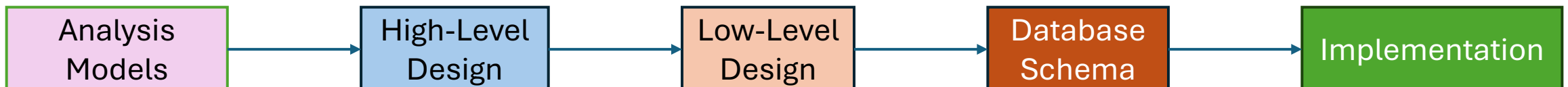
- Explain the transition from analysis models to system design.
- Distinguish High-Level Design, Low-Level Design, and Database Design.
- Identify main system components, modules, layers, data stores, external services, and security boundaries.
- Sketch a simple high-level architecture diagram from use cases, DFD, ERD, and business rules.

Low-Level Design

- Explain the purpose and deliverables of low-level design.
- Define component responsibility, internal flow, processing rules, validation rules, interface contracts, controls, and error handling.
- Use sequence diagrams to describe detailed component interactions.

Database Design

- Explain the relationship between Concept ERD and relational schema.
- Map entities, attributes, and relationships into relational tables.
- Identify primary keys, foreign keys, associative tables, constraints, and basic normalization issues.
- Prepare a simple data dictionary and database design checklist



PART 1

High-Level Design

A solid orange horizontal bar is positioned below the title text.

From Analysis to Design

Analysis asks:

- What does the system need to do?
- What problems must be solved?

Design asks:

- How should the system be organized?
- What components are needed?
- How will users, modules, data, and external systems interact?

What is Design Phase

- The design phase translates requirements into a blueprint for building the system.

It includes:

- System architecture
- Database design
- Interface design
- Form and report design
- Controls and security
- Deployment considerations

According to common ISAD design topics, the design phase includes system architecture, database design, interface design, forms and reports, controls, and deployment considerations.

“Design activities are related to each other; database, interfaces, controls and component design should be checked for consistency.

High-Level vs Low-Level

High-Level Design	Low-Level Design
Big picture	Detailed specification
Architecture and modules	Algorithms and validation rules
Main data groups	Table fields and constraints
Major interfaces	Screen details
Used for planning and communication	Used for implementation

What High-Level Design Answers

High-Level Design answers:

- What are the main system components?
- What modules are needed?
- Where is data stored?
- What external systems are involved?
- How do users access the system?
- What security boundaries exist?

High-Level Design Inputs and Outputs

Inputs from Analysis

- Use case diagram
- DFD
- ERD
- Requirements list
- Business rules
- BPMN workflow

Outputs of Design

- Architecture diagram
- Module list
- Database overview
- Interface overview
- Security/control plan
- Key screen list

Architecture View

Software architecture can be viewed from multiple perspectives: code organization, runtime interaction, and physical environment/deployment.

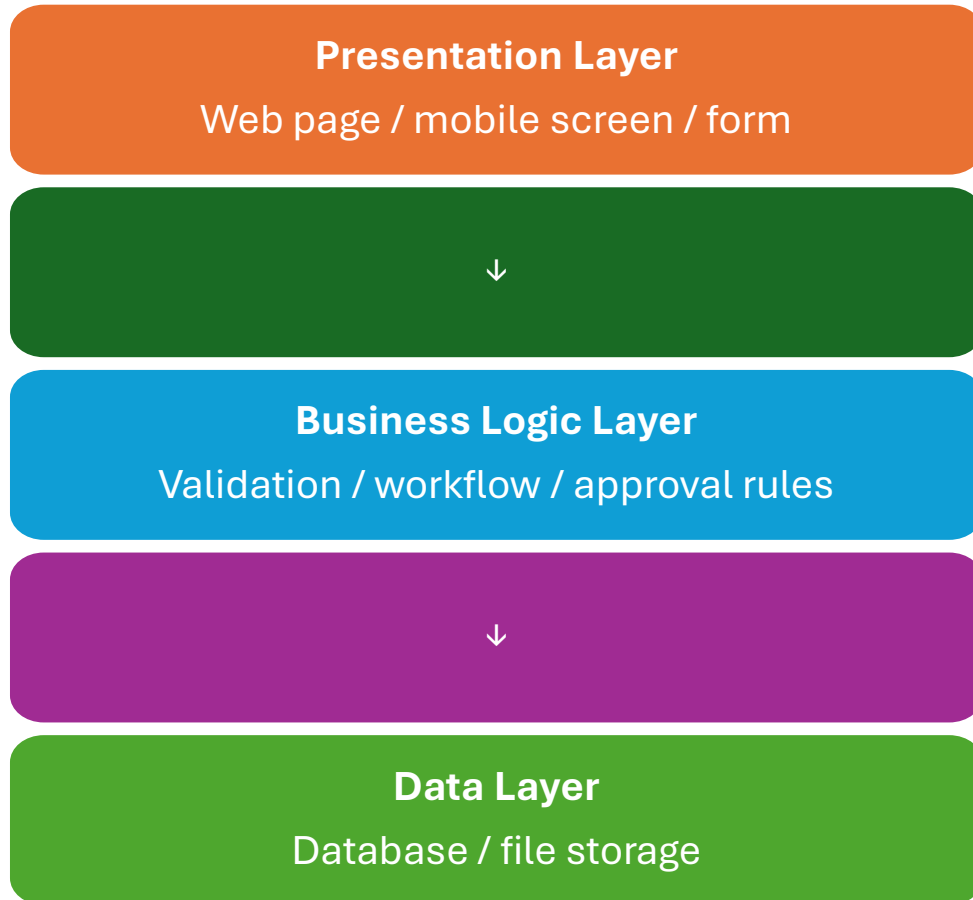
We use three simple views:

1. **Module View** — What parts the system has
2. **Component/Connector View** — How parts interact
3. **Deployment View** — Where parts run

A high-level architecture diagram shows the main users, presentation layer, application modules, database, and external services. It provides a big-picture view of how the system is organized.

In this course, students only need a simple high-level architecture view - mainly means a simple layered architecture diagram and module responsibility list.

Layered Architecture



Users should not directly access the database. They interact through the presentation layer and application logic.

Why useful?

- Easier to maintain
- Clear responsibility
- Supports testing
- Reduces dependency

Example: Academic Service Request System

Main modules:

- User Management Module
- Request Module
- Document Module
- Review Module
- Notification Module
- Reporting Module

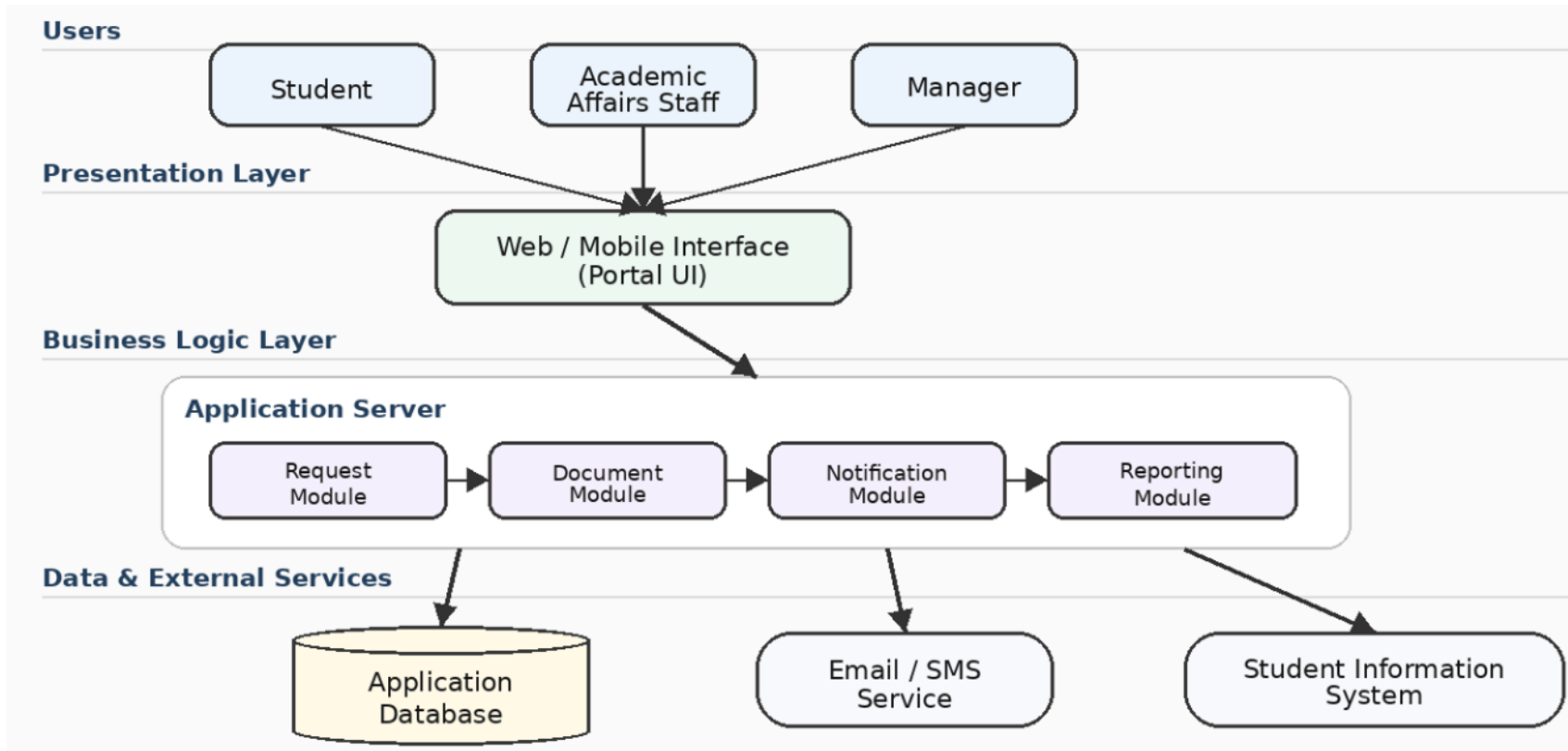
Main data:

- Student
- Service Request
- Uploaded Document
- Staff Review
- Notification
- Status History

External system:

- Email/SMS service
- Student information system

High-Level Architecture Diagram



Actors access the system through the presentation layer. Business modules handle rules and workflows. Data layer stores business data. External services support notification or integration.

The architecture diagram shows layers and system components, not step-by-step workflow.

Module Design

A module is a system part responsible for a related group of functions.

Good module design should have:

- Clear purpose
- Clear input/output
- Limited responsibility
- Low dependency on other modules

Example:

Request Module

- Create request
- Update request status
- Retrieve request history

Module	Responsibility	Input	Output
Request Module	Manage request creation and status	Request form	Request record/status
Document Module	Store uploaded documents	File upload	Document record
Notification Module	Send status messages	Student ID, status	Email/SMS notification

Interface Between Modules

An interface defines how modules communicate.

Example:

Request Module → Notification Module

Input:

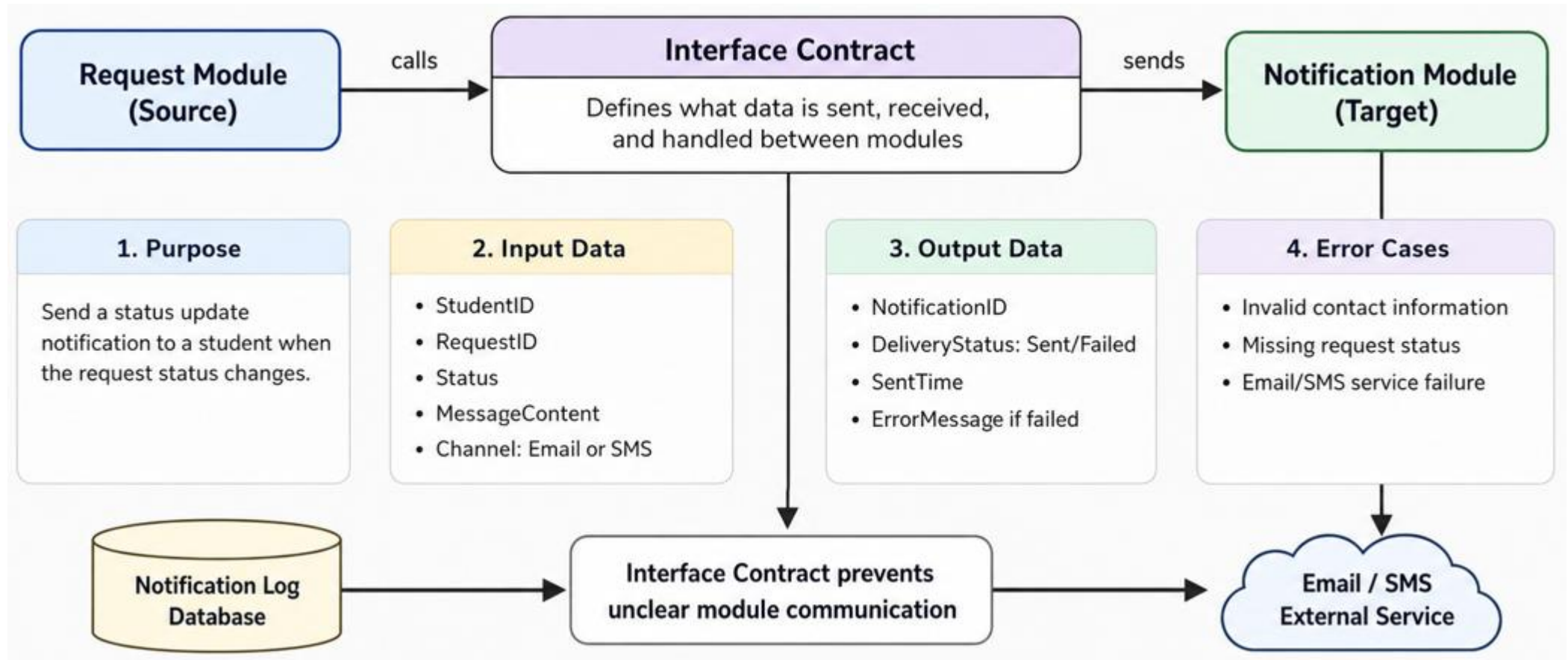
- Student ID
- Request ID
- Status
- Message content

Output:

- Notification sent / failed
- Timestamp

Interface design reduces integration problems because each module knows what data it sends and receives.

Interface Between Modules

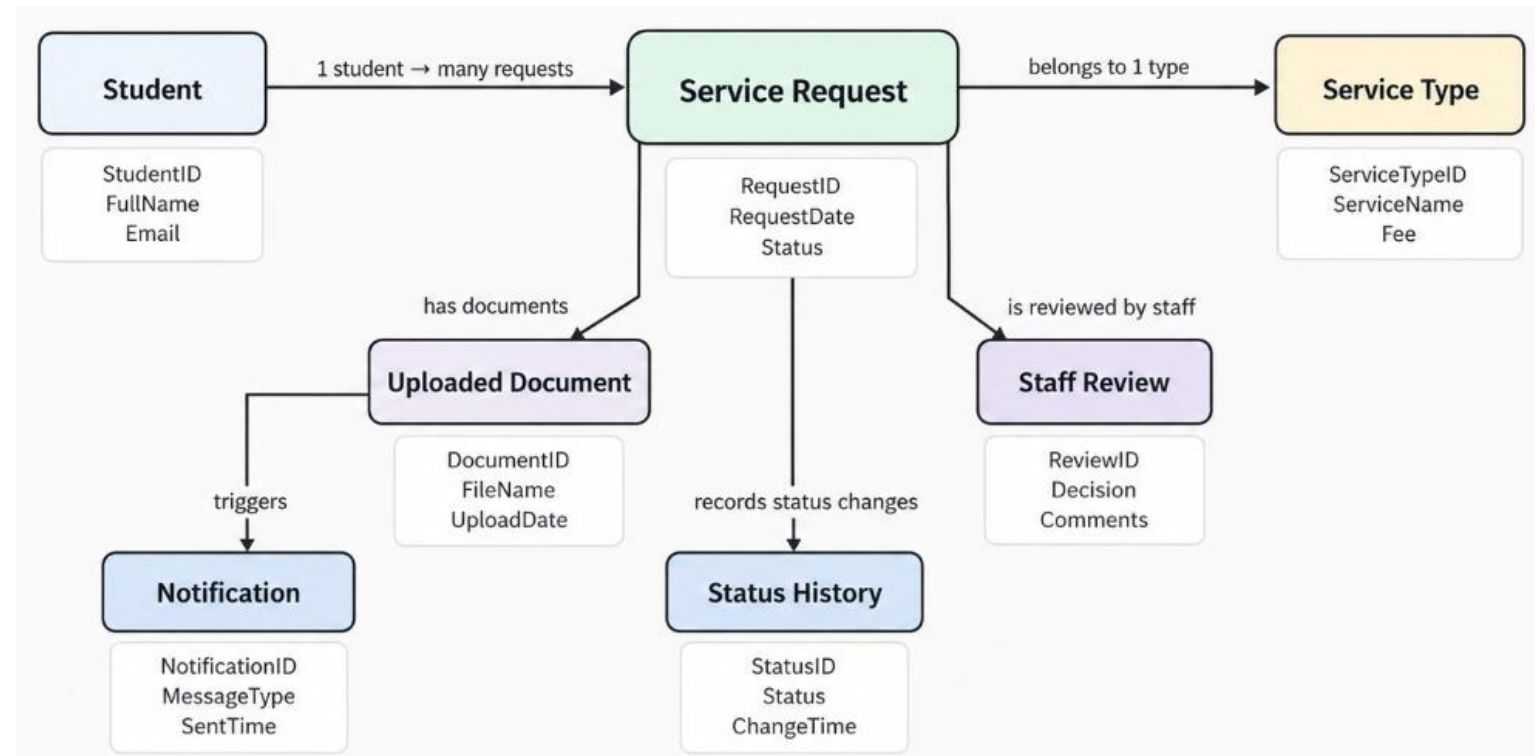


High-Level Data Design

High-level data design identifies the major data groups and their relationships. At this stage, the diagram does not need to show detailed database fields, constraints, or SQL implementation.

At HLD level, students identify major data groups only. Detailed tables, keys, constraints, and normalization are handled in Database Design.

Example:



Interface, Form, and Report Connection

A design specification should include:

- Narrative overview
- Sample design
- Testing and usability assessment

For High-Level Design, we do not need detailed UI yet, but we should identify:

- **Main screens:** Login Page, Student Dashboard, Staff Dashboard, Status Tracking Screen
- **Main forms:** Academic Document Request Form, Document Upload Form, Staff Review Form
- **Main reports:** Request Status Report, Monthly Service Request Report, Pending Request Report

Interface and Dialogue Design

Interface design focuses on how information is provided to and captured from users, while dialogue design focuses on the sequence of displays.

For High-Level Design, include:

- Login page
- Request form
- Staff dashboard
- Review screen
- Status tracking screen

Security and Controls

High-Level Design should consider:

- Authentication: Who is the user?
- Authorization: What can the user do?
- Data validation: Is input valid?
- Audit trail: Who changed what and when?
- Backup and recovery: How is data protected?

Control	Example
Authentication	Student logs in with university account
Authorization	Student can view only own requests
Data validation	Required documents cannot be empty
Audit trail	Store who approved/rejected request
Backup	Daily backup of request and document records

High-Level Design

Rules and steps to draw diagram

Rules:

1. Show the system structure, not workflow steps.
2. Organize components into layers: Users, Presentation, Application, Data, External Services.
3. Users must access the system through the interface, not directly through the database.
4. Application modules should represent major responsibilities, not individual actions.
5. Data stores and external services should be shown separately.
6. Use arrows to show communication or dependency, not strict time order.
7. Keep the diagram simple, readable, and implementation-oriented.

Steps:

- Step 1: Identify users
- Step 2: Identify the main interface
- Step 3: Identify the application layer
- Step 4: Identify major modules
- Step 5: Identify data stores and external services
- Step 6: Draw the connections

Sketch a High-Level Architecture Diagram

Scenario: Student Academic Document Request System

A student uses an online portal to submit an academic document request. Staff members review submitted requests, check uploaded documents, update request status, and notify students about the result.

Task:

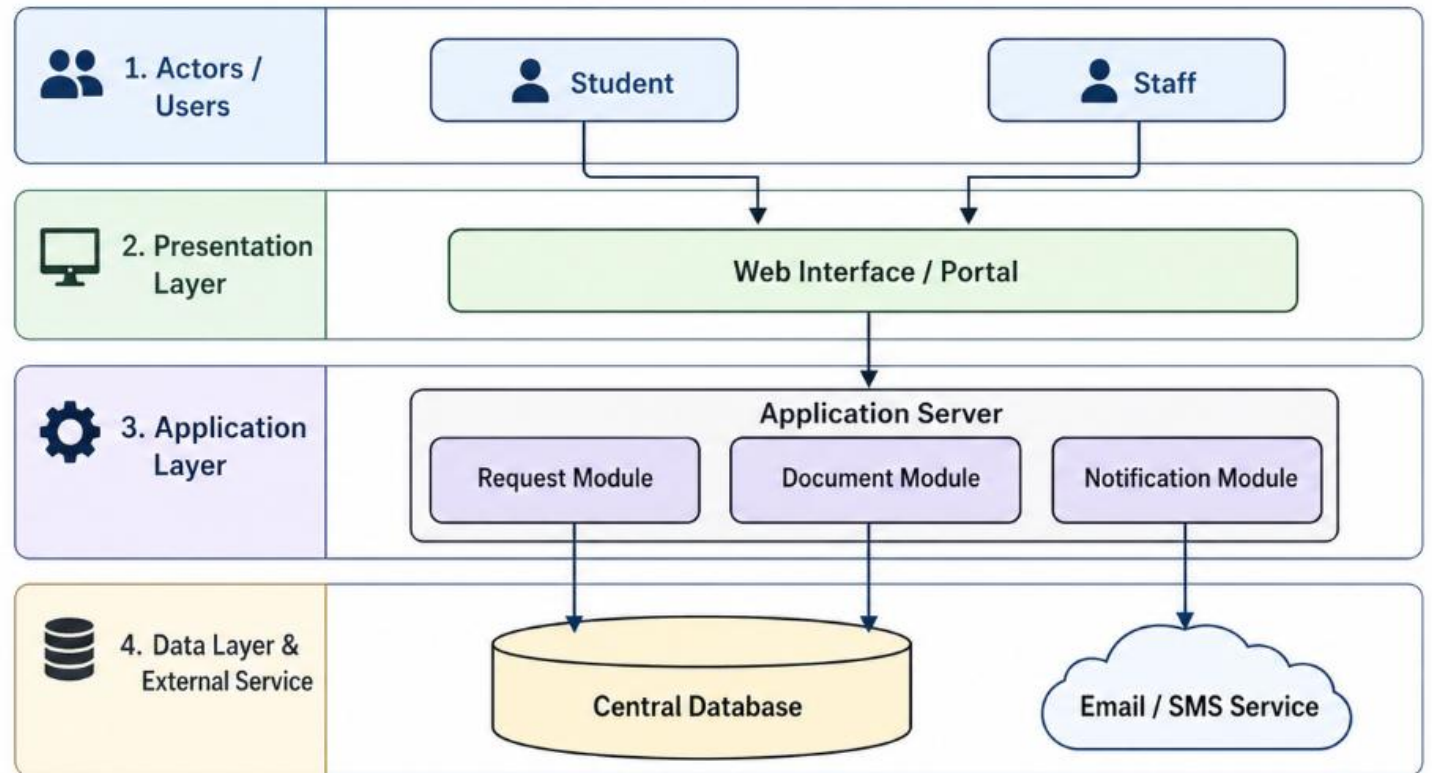
- Sketch a high-level architecture diagram for the system.
- Do not draw workflow steps such as Submit → Review → Approve. Draw system components and their relationships.

Your diagram should include:

Layer	Components
Actors	Student, Staff
Presentation Layer	Web Interface / Portal
Application Layer	Application Server
Business Modules	Request Module, Document Module, Notification Module
Data Layer	Database
External Service	Email / SMS Service

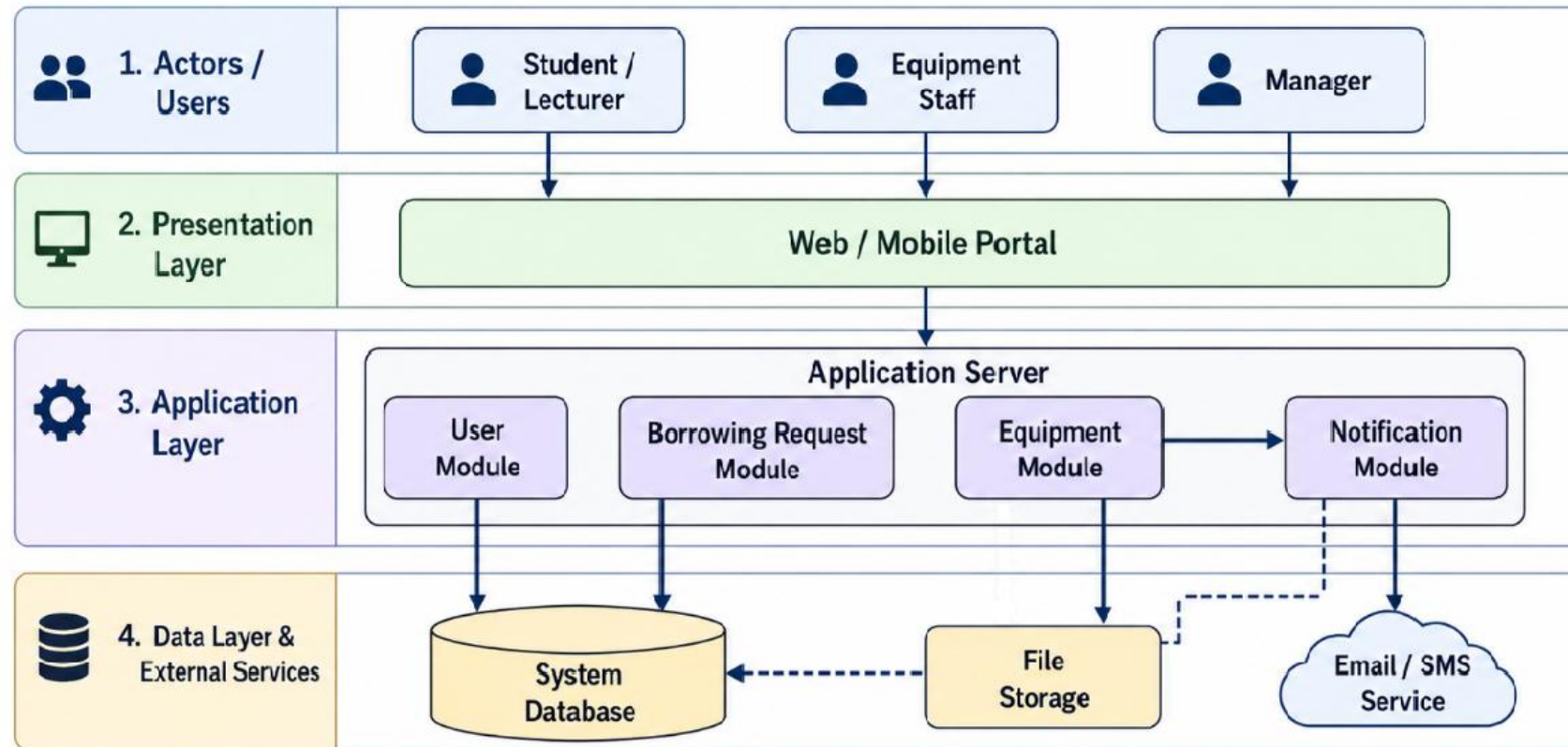
Sample Answer

This sample answer shows a simple layered architecture from users to the web portal, application server, modules, database, and external notification service



Practice: Campus Equipment Borrowing System

The high-level architecture diagram



This diagram show the system structure, not step-by-step workflow.

Part 1 Summary

High-Level Design gives the system big picture.

Students should remember:

- Architecture shows major components
- Modules divide responsibilities
- Interfaces define communication
- Data design identifies major data groups
- Security and usability must be considered early

HLD is not a workflow diagram and not a database schema

PART 2

Low-Level Design

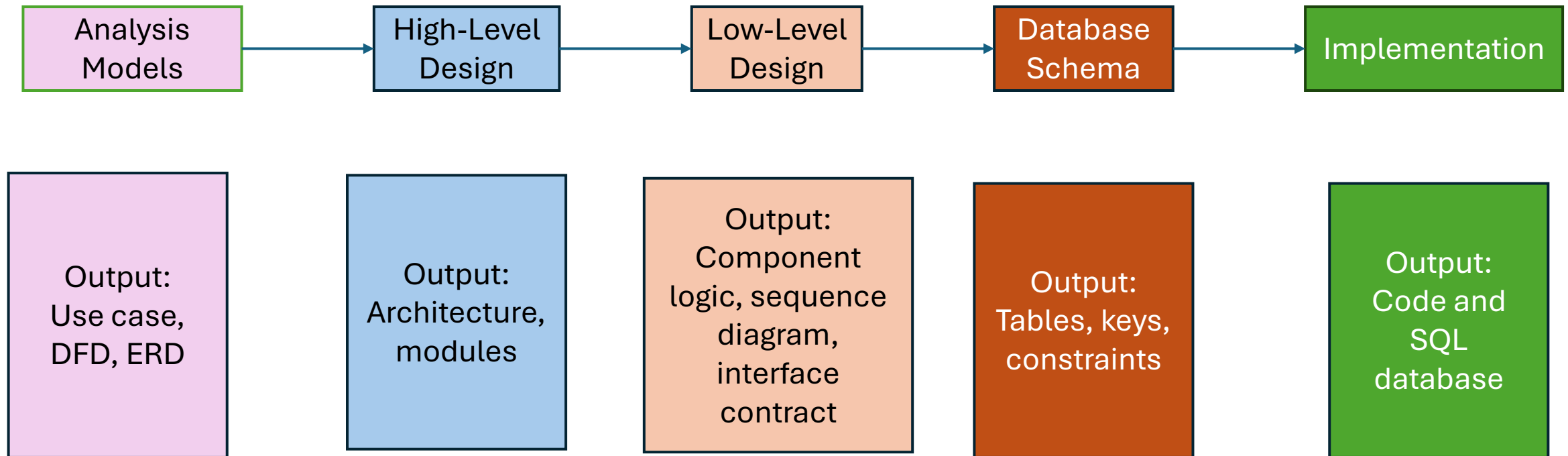
A solid orange horizontal bar is positioned below the title text.

From High-Level Design to Low-Level Design

- High-Level Design defines the overall system structure.
- Low-Level Design explains how each component works internally.

High-Level Design	Low-Level Design
Defines main modules	Defines internal component logic
Shows overall architecture	Shows detailed interactions
Answers “What modules exist?”	Answers “How does each module work?”
Output: architecture / module structure	Output: component design, internal flow, processing rules, validation rules, interface contract, sequence diagram

From High-Level Design to Low-Level Design



What Is Low-Level Design?

Low-Level Design (LLD) specifies how each system component works internally, how it communicates with other components, and how business rules are enforced before implementation.

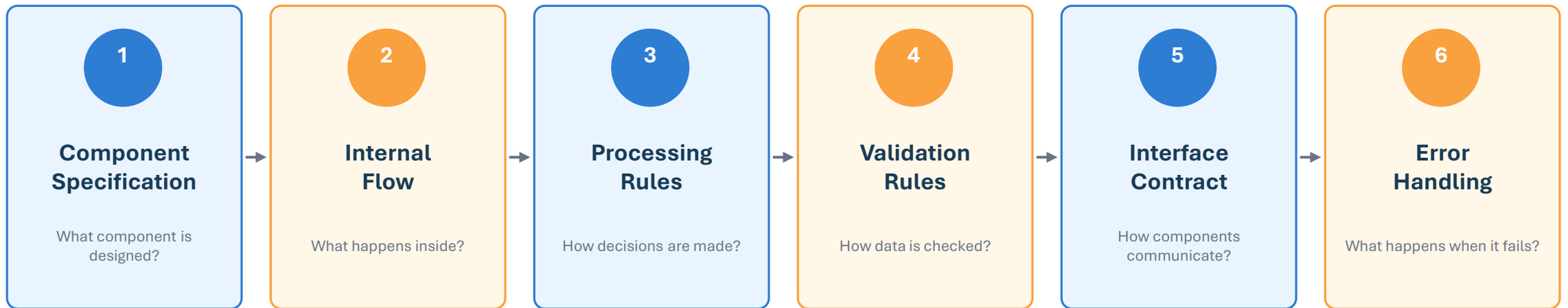
Key terms:

- Component logic
- Interface contract
- Validation rule
- Data rule
- Detailed interaction
- Exception handling

Low-Level Design Deliverables

Deliverable Group	Examples	Purpose
Component Design	Component specification, processing rules, validation rules	Define what each component does
Interaction Design	Interface contract, sequence diagram, error handling	Define how components communicate
Implementation Support	Data access rules, UI behavior, acceptance criteria	Support coding and testing

Low-Level Design Deliverables



A sequence diagram is not enough.

A complete LLD must also explain the component responsibility, internal logic, rules, data contract, and failure response.

Low-Level Design in the Design Phase

In the design phase, **analysts** and **designers refine**:

- Process requirements into component behavior.
- Data requirements into database structure.
- Interface requirements into screens and interactions.
- Control requirements into validation, authorization, logs, and recovery rules.

Key idea:

Low-Level Design should be detailed enough for implementation, but it should not become actual source code.

Example:

- Use case → What steps happen? → Sequence diagram → Developer logic
- ERD → What tables are needed? → Relational schema → SQL tables

LLD specifies logic, rules, data access, and interactions, while source code implements those specifications in a programming language

Component-Level Thinking

A component is a part of the system responsible for a specific function.

Good component design should define:

- Responsibility
- Input data
- Output data
- Business rules
- Dependencies
- Error cases

Example dependencies:

- Request Module depends on Request Database.
- Notification Module depends on Email/SMS Service.
- Document Upload Module depends on file storage.

Example error cases:

- Missing required document
- Invalid file type
- Email/SMS sending failed

Good vs. Poor Component Design

Poor Design	Better Design
Request Module sends email directly	Request Module calls Notification Module
Request Module stores uploaded files directly	Document Module handles file storage
Request Module generates reports	Reporting Module handles reports
One module does too many things	Each module has one clear responsibility

Component Responsibility

Each component should have one clear responsibility.

Keep components cohesive and avoid unnecessary dependency.

Example: Request Module

- Creates a new document request.
- Updates request status.
- Stores request information.
- Sends request status to the Notification Module.

Not good:

Request Module directly sends email, manages staff accounts, stores uploaded files and generates reports.

Better design:

- Request Module manages request logic.
- Notification Module handles message sending.
- Document Module handles uploaded files.
- Reporting Module handles reports.

Low-Level Design Template

Component Information

- Component Name
- Purpose
- Related Use Case
- Database Tables Used
- Other Components Called

Behavior Specification

- Input Data
- Output Data
- Processing Rules
- Validation Rules
- Error Cases
- Security / Control Rules
- Diagram

Example:

Component Name: Request Module

Input Data: StudentID, TypeID, Purpose, UploadedDocument

Output Data: RequestID, Status

Other Components Called: Validation Service, Notification Module

Component Specification

Define the selected component clearly before drawing the detailed interaction.

What to write

Specification Item	Example
Component Name	Request Submission Component
Purpose	Receive, validate, and save a student document request
Related Use Case	Submit Academic Document Request
Inputs	StudentID, TypeID, Purpose, UploadedDocument
Outputs	RequestID, Status, ConfirmationMessage
Dependencies	Validation Service, Request Database, Notification Module
Data Used	STUDENT, DOCUMENT_TYPE, DOCUMENT_REQUEST, UPLOADED_DOCUMENT

Design principle

**ONE
clear
responsibility**

Good component specification avoids
“one module does everything”.

Use it before writing rules or drawing
sequence diagrams.

Mini Example

Component Name: Document Request Module

Purpose: Manage the submission and status update of academic document requests.

Related Use Case: Submit Document Request

Input Data: StudentID, TypeID, Purpose, UploadedDocument

Output Data: RequestID, Status, ConfirmationMessage

Processing Rules:

- Request cannot be submitted without a document type.
- Status is set to Submitted after successful validation.
- Duplicate request on the same day is not allowed.

Validation Rules:

- TypeID is required.
- Uploaded file must be PDF, JPG, or PNG.
- Purpose cannot exceed 255 characters.

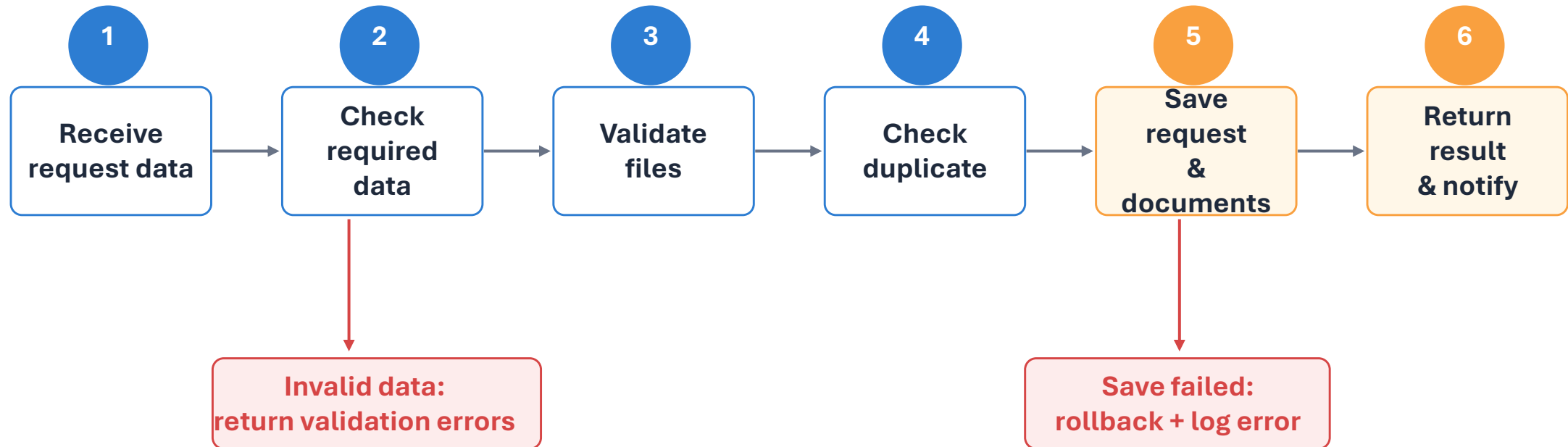
Error Cases:

- Missing document type.
- Invalid file type.
- Database save failure.

Security Rules:

- Student can create only their own requests.
- Staff can review assigned requests only.

Internal Flow



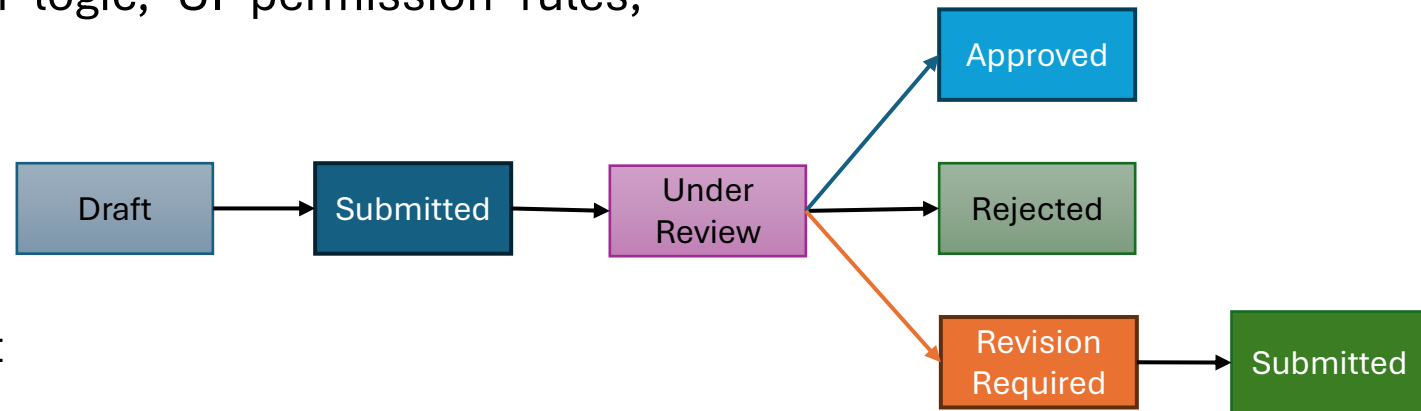
Internal Flow answers “what happens inside the component”; Sequence Diagram answers “which participant sends which message over time”.

Low-Level Processing Rules

Processing rules define how the system handles data and decisions. Processing rules may become validation logic, UI permission rules, database constraints, or test cases.

Example: Document Request Rules

- Request cannot be submitted without document type.
- Required attachments depend on document type.
- Request status must follow valid transition:
 - Draft → Submitted
 - Submitted → Under Review
 - Under Review → Approved / Rejected / Revision Required
- Student can revise only when status is Revision Required.



Rule example

- Student can revise only when status = Revision Required
- Approved and Rejected are terminal business outcomes.
- Revision Required allows the student to revise and resubmit.

Design Implication

Processing rules become validation logic, UI permissions, and database constraints.

Processing Rules

Define what the system does.

Rule ID	Condition	System Action	Design Implication
PR-01	After successful validation	Create request and set Status = Submitted	Application logic + database default
PR-02	Same student, same document type, same day	Do not create duplicate request	Business rule check before save
PR-03	Staff marks request incomplete	Set Status = Revision Required	Status transition rule
PR-04	Request is Approved or Rejected	Student cannot revise it	UI permission + service rule
PR-05	Revision Required and student resubmits	Keep original RequestID and add new file version	Data access + audit trail

How to write rules

Condition

→

Action

→

Design Control

Rules should be testable.
Avoid vague verbs such as
“process request”.

Validation Rules

Define what data is acceptable.

Define how the system prevents incorrect, incomplete, or unauthorized data.

Required

TypeID, Purpose,
UploadedDocument must not be empty.

Format

Email format; file extension must be PDF, JPG, or PNG.

Range / Length

Purpose $\leq$ 255 characters; file size $\leq$ allowed maximum.

Business Rule

No duplicate request for the same student and document type on the same day.

File Control

Scan file, store FileURL, and reject unsupported content.

Authorization

Student can submit only for own StudentID; staff can review assigned requests.

Output of this section: a validation table with Field / Rule / Error Message / Where Enforced.

Interface Contract Between Components

An interface contract defines how components communicate.

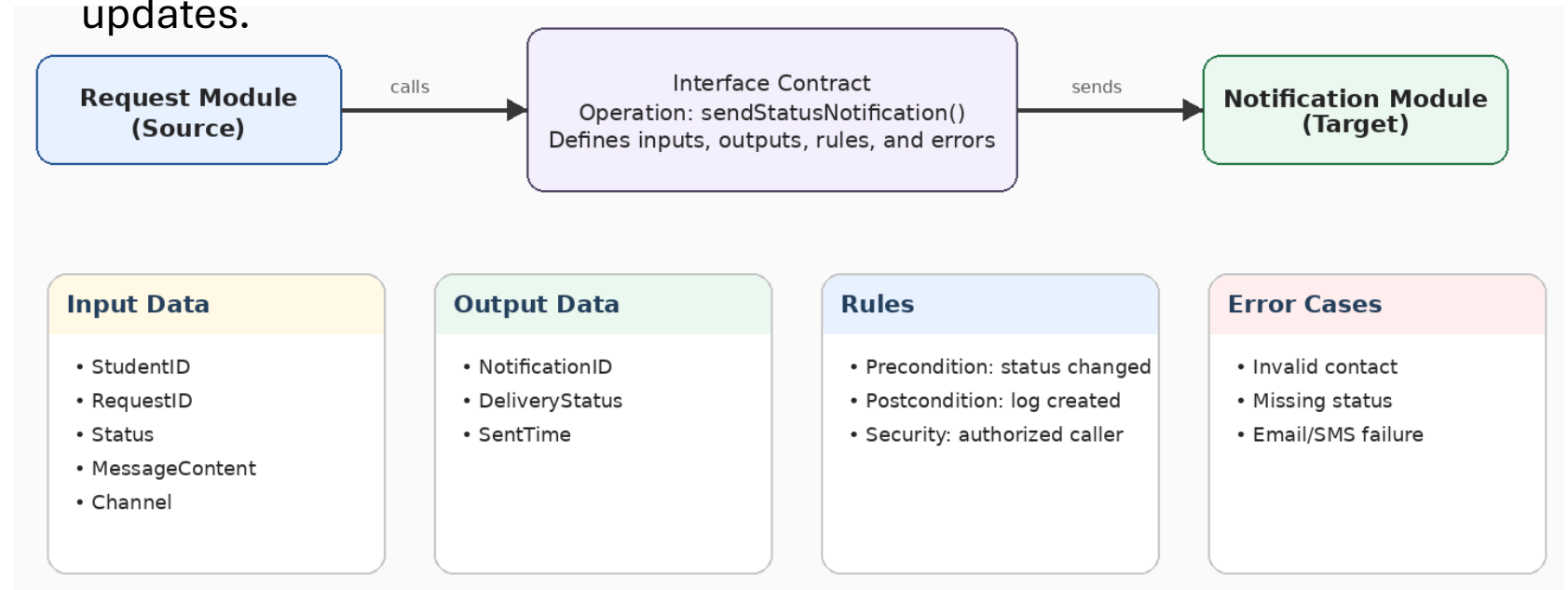
An interface contract prevents unclear communication between modules.

A **complete contract** should **specify**:

- Source component
- Target component
- Operation name
- Input data
- Output data
- Precondition
- Postcondition
- Error cases
- Data format
- Security requirement

Example:

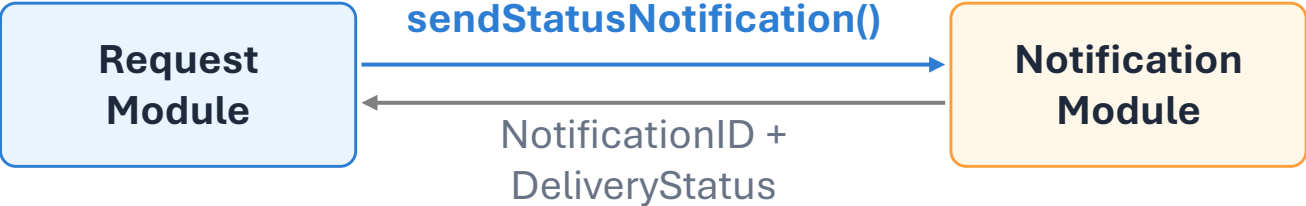
Request Module calls Notification Module to send request status updates.



An interface contract reduces integration errors because both source and target components agree on the required input, expected output, and error cases.

Interface Contract Template

Specify how one component calls another component.



Operation example

`sendStatusNotification(studentId, requestId, status, messageContent, channel)`

Precondition: request status has changed

Postcondition: notification record is created

Error cases: invalid contact, missing status, service timeout

Contract Item	Specification
Source	
Target	
Input	
Output	
Format	
Security	

Interface Contract Example

Operation:

```
sendStatusNotification  
(  
    studentId: string,  
    requestId: string,  
    status: string,  
    messageContent:  
    string,  
    channel: string  
)
```

Possible Channel values:
Email, SMS, In-App

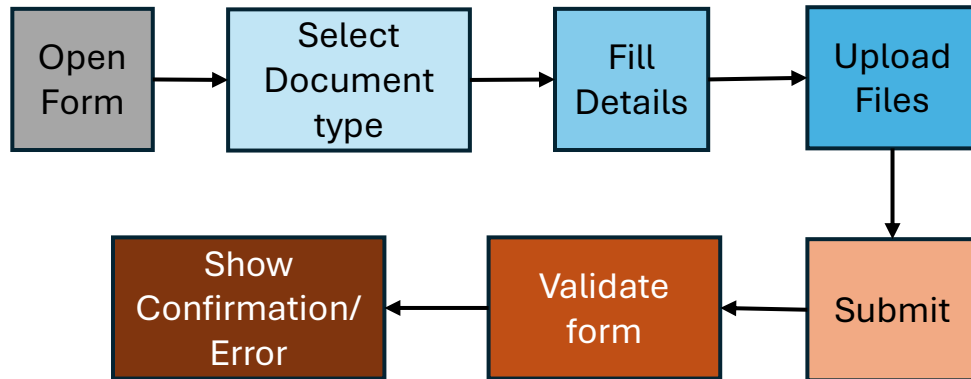
Possible DeliveryStatus values: Sent, Failed, Pending

Item	Specification
Source	Request Module
Target	Notification Module
Input	StudentID, RequestID, Status, MessageContent, Channel
Output	NotificationID, DeliveryStatus, SentTime
Precondition	Request status has changed
Postcondition	Notification record is created
Error cases	Invalid contact, missing status, email/SMS failure
Data format	JSON object / structured message
Security	Only authenticated internal modules can call this operation.

User Interaction Details in Low-Level Design

Low-level design may also describe how users interact with screens.

Example: Submit Document Request



Comparison table

Aspect	Interface Design	Dialogue Design
Focus	What users see and enter	How users move between screens
Includes	Fields, buttons, messages	Screen sequence, navigation, user actions
Example	Document Type field, Submit button, error message	Open Form → Fill Details → Submit → Confirmation/Error
LLD Output	Screen specification	User interaction flow

Interface contract is for communication between system components.

Interface/dialogue design is for communication between the user and the system

Sequence Diagram for Component Interactions

A sequence diagram shows how actors and system components interact over time.

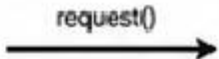
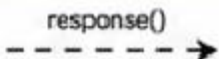
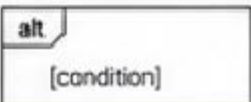
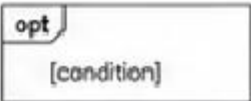
Main elements:

- Actor
- Object / component
- Lifeline
- Activation bar
- Message
- Return message
- Alternative fragment alt
- Loop fragment loop

Use when:

- Explaining one use case in detail.
- Showing the order of system calls.
- Clarifying component responsibilities.

Sequence Diagram Symbols

Notation	Name	Description	Example	
	Actor / Object	A participant in the interaction.		External user or role
	Lifeline	Shows the existence of an object over time.		Participant over time
	Activation (Execution Occurrence)	Represents the period when an object is performing an action.		Period of execution
	Message	A request or communication from one object to another.		Call or request
	Return Message	A response returned to the sender.		Return result
	alt Fragment	Represents alternative flows.		Conditional branches
	opt Fragment	Represents an optional flow.		Optional behavior
	loop Fragment	Represents repeated actions.		Repeated behavior

Reading a Sequence Diagram

Read a sequence diagram:

- From left to right: participants involved.
- From top to bottom: time order.
- Each arrow is a message or method call.
- Each **alt** block represents a condition.
- Each return message shows the result of a call.

Note: A sequence diagram is not a flowchart. It shows messages between participants over time.

Example condition:

- If request information is valid → create request.
- If request information is invalid → show error message.

Common Mistakes in Sequence Diagrams

- Drawing a sequence diagram like a flowchart.
- Missing important participants/components.
- Letting the actor directly access the database.
- Not showing alternative flows with alt.
- Using vague messages such as “process data”.
- Forgetting return messages for important results.

Error Handling and Feedback

Low-level design must define how the system responds when something goes wrong.

Good error handling should:

- Explain what happened.
- Tell the user what to correct.
- Avoid technical messages.
- Log technical errors for support staff.
- Preserve user-entered data when possible.

Example:

- Bad: “Error code 500.”
- Good: “The system could not upload the file. Please check the file format and try again.”

User message:

- The file could not be uploaded. Please check the file format and try again.

System log:

- Upload failed because file size exceeded the maximum limit.

Error Handling Matrix

Design user feedback, system action, and logging before coding.

Error Case	User Message	System Action	Log / Audit
Missing document type	Please select a document type.	Stop submission; keep entered data	Validation warning
Unsupported file format	The file type is not supported. Please upload PDF, JPG, or PNG.	Reject file; allow re-upload	File validation event
Duplicate request	You already submitted this request today.	Do not create new record	Business rule event
Database save failure	The request could not be saved. Please try again later.	Rollback transaction; alert support	Technical error + trace ID
Notification failed	Your request was submitted, but notification delivery is pending.	Save notification as Pending; retry later	Integration error

Good error handling is not only “show an error”; it must also protect data, guide the user, and support technical recovery.

Controls in Low-Level Design

Controls protect system accuracy, security, and reliability.

Control	Example in Document Request System
Authentication	Student logs in with university account
Authorization	Student can view only own requests
Input control	Required documents cannot be empty
Processing control	Status must follow allowed transition
Audit log	Store who approved/rejected request
Backup and recovery	Daily backup of request records

Session 1 Practice Scenario

Scenario: Student Academic Document Request System

A student submits a request for an academic document, such as a transcript or enrollment certificate. The student fills in a request form, uploads supporting documents, and submits the request. The system validates the request information. If the request is valid, the Request Module saves the request and asks the Notification Module to send a confirmation to the student. Later, an academic affairs staff member reviews the request. If the request is incomplete, the staff member marks it as “Revision Required,” and the system sends a notification to the student. The student revises the request and uploads the missing document again.

Task: Prepare a Low-Level Design for the Request Submission Component.

Must include:

1. Component Specification
2. Internal Flow
3. Processing Rules
4. Validation Rules
5. Interface Contract
6. Error Handling

Practice:

Sequence Diagram Reading Guide

Expected participants/components:

- Student
- Web Interface
- Request Module
- Validation Service
- Request Database
- Notification Module
- Email/SMS Service
- Academic Affairs Staff

Suggested interaction flow:

- Student submits request through Web Interface.
- Web Interface sends request data to Request Module.
- Request Module calls Validation Service.
- If invalid, Web Interface shows validation error.
- If valid, Request Module saves request to Request Database.
- Request Module sends notification request to Notification Module.
- Notification Module sends message through Email/SMS Service.
- Staff reviews request.
- If revision is needed, Request Module updates status.
- Notification Module sends revision notification to Student.

Suggested Answer:

Component Specification

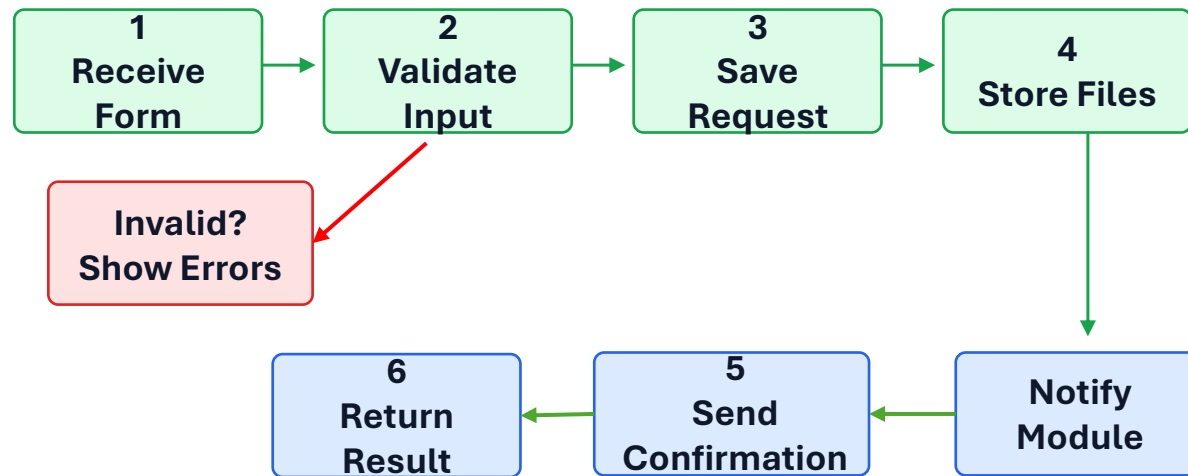
1. Component Specification

Component Name	Request Submission Component
Purpose	Receive, validate, save academic document requests and trigger confirmation.
Related Use Case	Submit Document Request
Input Data	StudentID, DocumentTypeID, Purpose, UploadedDocument(s)
Output Data	RequestID, Status = Submitted, ConfirmationMessage
Tables Used	STUDENT, DOCUMENT_TYPE, DOCUMENT_REQUEST, UPLOADED_DOCUMENT
Dependencies	Validation Service, Request Database, Notification Module
Security Scope	Authenticated student can create only their own request.

Suggested Answer: Internal Flow

2. Internal Flow

Internal flow explains the logic inside one component. It is more detailed than “draw a sequence diagram”.



Design decision

- The component does not send email directly; it calls the Notification Module.
- Validation happens before database save.
- A failed notification should not delete a successfully saved request.

Suggested Answer: Processing Rules

ID	Rule	Design Implication
P1	Student must be authenticated before submitting a request.	Check session/token before processing.
P2	Request is created only after required fields and files pass validation.	Do not write incomplete request to database.
P3	Initial request status is set to Submitted.	Use default status after successful save.
P4	Duplicate active request for same document type on the same day is not allowed.	Search existing requests before insert.
P5	Required supporting documents depend on document type.	Load document-type rules before validation.
P6	After save, confirmation notification is requested.	Call Notification Module using interface contract.

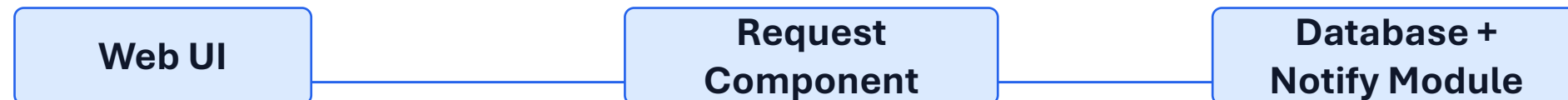
Suggested Answer: Validation Rules

ID	Validation	Condition	User Message
V1	Document type required	TypeID is empty	Please select a document type.
V2	Purpose length	Purpose > 255 chars	Purpose must be shorter than 255 characters.
V3	File type	Not PDF/JPG/PNG	Only PDF, JPG, or PNG files are allowed.
V4	File size	File > 10MB	File size must not exceed 10MB.
V5	Ownership	StudentID differs from login user	You can submit only your own request.
V6	Required attachment	Required file missing	Please upload the required supporting document.

Suggested Answer: Interface Contract

Operation: submitDocumentRequest(requestPayload)

Source	Web Interface
Target	Request Submission Component
Request	studentId, typeId, purpose, files[]
Response	requestId, status, confirmationMessage
Precondition	Student is authenticated; request form is open.
Postcondition	Request and documents are saved; notification request is created.
Security	HTTPS/session token; student can create only own request.
Error Response	errorCode, userMessage, technicalLogId



Suggested Answer: Error Handling

Code	Error Case	User/System Response	Log/Recovery
E01	Missing type	Show field-level message and keep form data.	No technical log needed.
E02	Invalid file	Reject upload and show allowed formats.	Log file metadata.
E03	Duplicate request	Stop submission and show duplicate warning.	Record validation failure.
E04	Database save failure	Show retry message; do not show technical details.	Log exception + request payload ID.
E05	Notification failure	Save request; mark notification as Pending/Failed.	Retry queue or support review.

Recommended answer principle: user messages should be understandable; technical details should go to logs, not to students.

Part 2 - Summary

Low-level design turns system architecture into detailed implementation guidance.

Remember:

- Component responsibility must be clear.
- Interface contracts define how components communicate.
- Sequence diagrams show detailed interactions over time.
- Validation and error handling must be designed before coding.
- Controls such as authorization and audit logs are part of good design.
- Good Low-Level design makes coding, testing, and maintenance easier.

PART 3

DATABASE DESIGN

A solid orange horizontal bar is positioned below the main title text.

Session 2 Overview: Database Design

This session covers:

- Meaning and purpose of database design.
- Logical versus physical database design.
- Relational database model.
- Primary keys and foreign keys.
- Transforming ERD into relational schema.
- Normalization basics.
- Constraints, data integrity, indexes, and performance.
- In-class database design activity.

What is Database Design?

- Database design is the process of organizing data so it can be stored, retrieved, updated, and protected effectively.
- It transforms data requirements into a structure that can be implemented in a DBMS.
- It should support both operational tasks and reporting needs.
- A good database design improves accuracy, consistency, performance, security, and future maintenance.

Academic note

Database design is part of the design phase, but it is based on analysis outputs such as ERDs, business rules, data dictionaries, forms, and reports.



Database Design in the Design Phase

Database design connects analysis models with implementation.

A database design normally produces:

- A logical database model.
- A set of well-structured relations or tables.
- Primary key and foreign key definitions.
- Data integrity constraints.
- Physical field specifications such as data type and field length.
- Index decisions for important searches and reports.

Example output

ROOM(RoomID PK, RoomName, Capacity, BuildingID FK)

Logical Database Design vs Physical Database Design

Logical database design

- Describes the data structure independent of detailed technology choices.
- Focuses on relations, attributes, keys, and normalization.

Physical database design

- Describes how the database will be implemented technically.
- Focuses on data types, field size, indexes, storage, access speed, and constraints.

Example

Logical: Student(StudentID, FullName, Email)

Physical: StudentID CHAR(10), FullName VARCHAR(100), Email VARCHAR(120) UNIQUE.

Conceptual, Logical, and Physical Design

Level	Main Question	Output
Conceptual	What data does the organization need?	ERD
Logical	How should data be structured relationally?	Relations / schema
Physical	How will it be implemented in DBMS?	SQL tables, data types, indexes

Example:

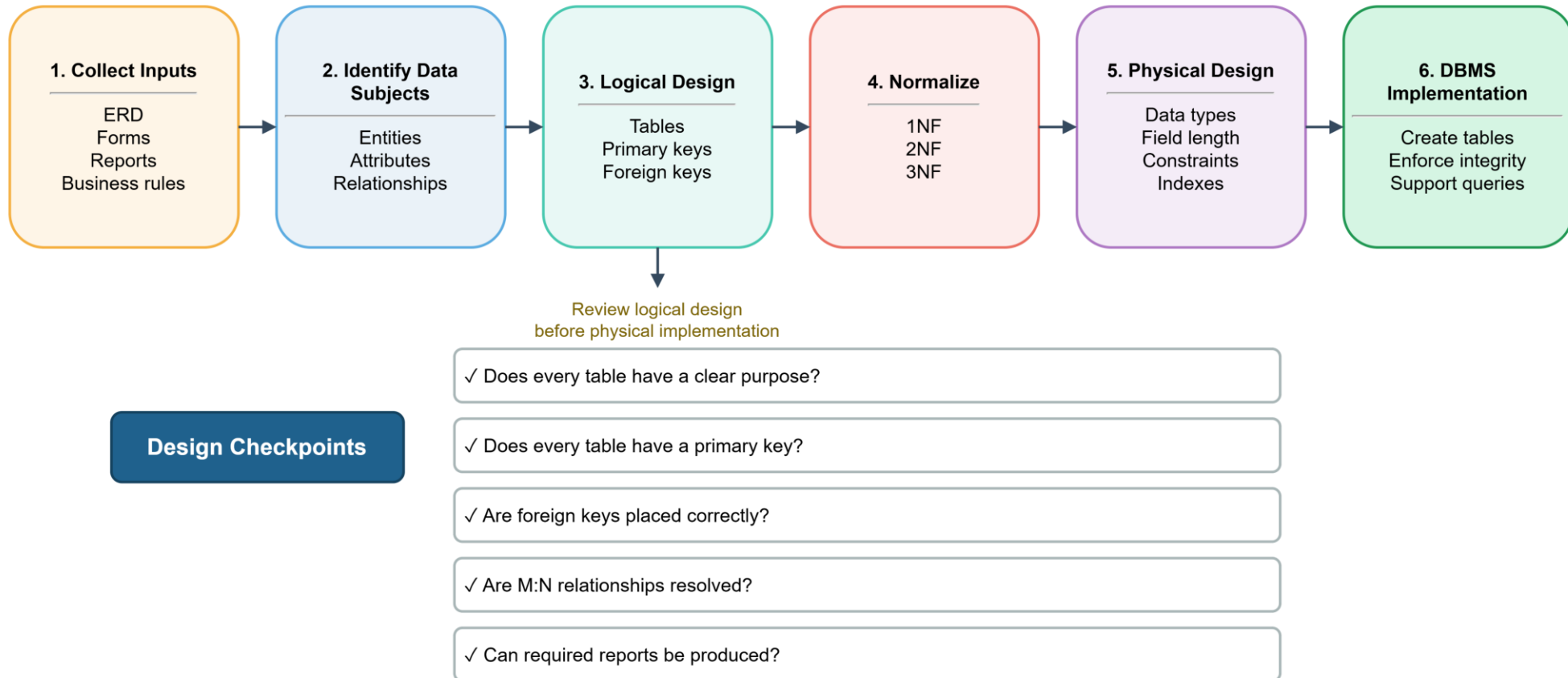
- Conceptual: Student submits Request.
- Logical: STUDENT and REQUEST relations.
- Physical: CREATE TABLE Student (...)

Conceptual design is technology-independent. Physical design depends on a specific DBMS.

Database Design Process

- Develop logical data model from forms and reports.
- Normalize user views.
- Integrate views into one logical model.
- Compare with ERD.
- Produce final logical database design.
- Specify physical database design.

Database Design Process



Inputs to Database Design

Database design should use:

- ERD and entity descriptions.
- Use cases and process models.
- Forms that collect data.
- Reports that display data.
- Business rules and validation rules.
- Security and access requirements.

Example

If a report must show monthly room utilization, the database must store booking date, room, time period, and booking status

Relational Database Model

In the relational model:

- Data is represented as relations, commonly implemented as tables.
- Each table contains rows and columns.
- Each row represents one record or instance.
- Each column represents one attribute or field.
- Relationships are represented using keys.

Simple example

STUDENT(StudentID PK, FullName, Email)

ENROLLMENT(EnrollmentID PK, StudentID FK, SectionID FK, EnrollDate)

Well-Structured Relations

A well-structured relation should:

- Represent one main subject.
- Avoid unnecessary data duplication.
- Use a primary key to identify each row.
- Store attributes that depend on the key.
- Reduce update, insert, and delete anomalies.

Bad example

Booking(BookingID, UserName, UserEmail, RoomName, BuildingName, BookingDate)

Problem

User and room information may be repeated in many booking rows.

Primary Keys

- A primary key uniquely identifies each row in a table.
- It must not be null.
- It should be stable and not change frequently.
- It may be a natural key or a surrogate key.

Examples

- Natural key: StudentID
- Surrogate key: BookingID

Academic note

A table without a clear primary key is difficult to update, reference, and control.

Foreign Keys

- A foreign key is an attribute in one table that refers to the primary key of another table.
- It implements relationships between tables.
- It supports referential integrity.

Example

ROOM(RoomID PK, RoomName, BuildingID FK)
BUILDING(BuildingID PK, BuildingName)

Meaning

Each room belongs to one valid building

Common Relationship Patterns

One-to-many relationship

- Put the primary key from the “one” side into the “many” side as a foreign key.

Many-to-many relationship

- Create an associative table.
- The associative table contains foreign keys from both related tables.

One-to-one relationship

- Usually combine into one table unless there is a strong reason to separate.

Transforming ERD to Relations: Entity Rule

Rule: Each strong entity usually becomes one table.

ERD concept Student entity with StudentID, FullName, Email, Phone.

Relational schema STUDENT(StudentID PK, FullName, Email, Phone)

Design check

- Does the table represent one subject?
- Does it have a primary key?
- Are the attributes directly related to the subject?

Transforming ERD to Relations: 1:M Rule

Rule: In a 1:M relationship, place the primary key of the “one” entity into the table of the “many” entity as a foreign key.

Example

One building has many rooms.

Schema

BUILDING(BuildingID PK, BuildingName)

ROOM(RoomID PK, RoomName, Capacity, BuildingID FK)

Meaning

Each room is assigned to one building; each building may contain many rooms.

Transforming ERD to Relations: M:N Rule

Rule: In an M:N relationship, create a new associative table.

Example A student can enroll in many sections. A section can have many students.

Schema

STUDENT(StudentID PK, FullName)

SECTION(SectionID PK, CourseID FK, Semester)

ENROLLMENT(StudentID FK, SectionID FK, EnrollDate, Status)

Possible PK PK(StudentID, SectionID) or EnrollmentID PK

Associative Entities

An associative entity is used when:

- A relationship has its own attributes.
- A many-to-many relationship must be resolved.
- The relationship itself becomes important to the system.

Examples

- Enrollment between Student and Section.
- OrderLine between Order and Product.
- BookingEquipment between Booking and Equipment.

ERD-to-Relational Mapping Summary

ERD Pattern	Mapping Rule	Relational Schema Example
Strong Entity	Each strong entity becomes one table.	STUDENT(<u>StudentID</u> PK, FullName, Email, Phone)
1:M Relationship	Place the PK from the “one” side into the table on the “many” side as FK.	BUILDING(<u>BuildingID</u> PK, BuildingName) ROOM(<u>RoomID</u> PK, RoomName, Capacity, BuildingID FK)
M:N Relationship	Create an associative table containing FKs from both related tables.	STUDENT(<u>StudentID</u> PK, FullName) SECTION(<u>SectionID</u> PK, CourseID, Semester) ENROLLMENT(StudentID FK, SectionID FK, EnrollDate, Status)
Relationship Attribute	Put attributes that describe the relationship into the associative table.	ORDER_LINE(OrderID FK, ProductID FK, Quantity, UnitPrice)

Functional Dependence

- Functional dependence means one attribute determines another attribute.
- If StudentID determines StudentName, then StudentName depends on StudentID.
- Functional dependence helps identify whether a table is well structured.

Example

StudentID $\rightarrow$ StudentName, Email

CourseID $\rightarrow$ CourseName, Credits

StudentID $\rightarrow$ StudentName, Email and CourseID $\rightarrow$ CourseName, Credits

Normalization

Normalization is a database design technique used to organize data into well-structured tables to reduce redundancy, avoid anomalies, and improve consistency.

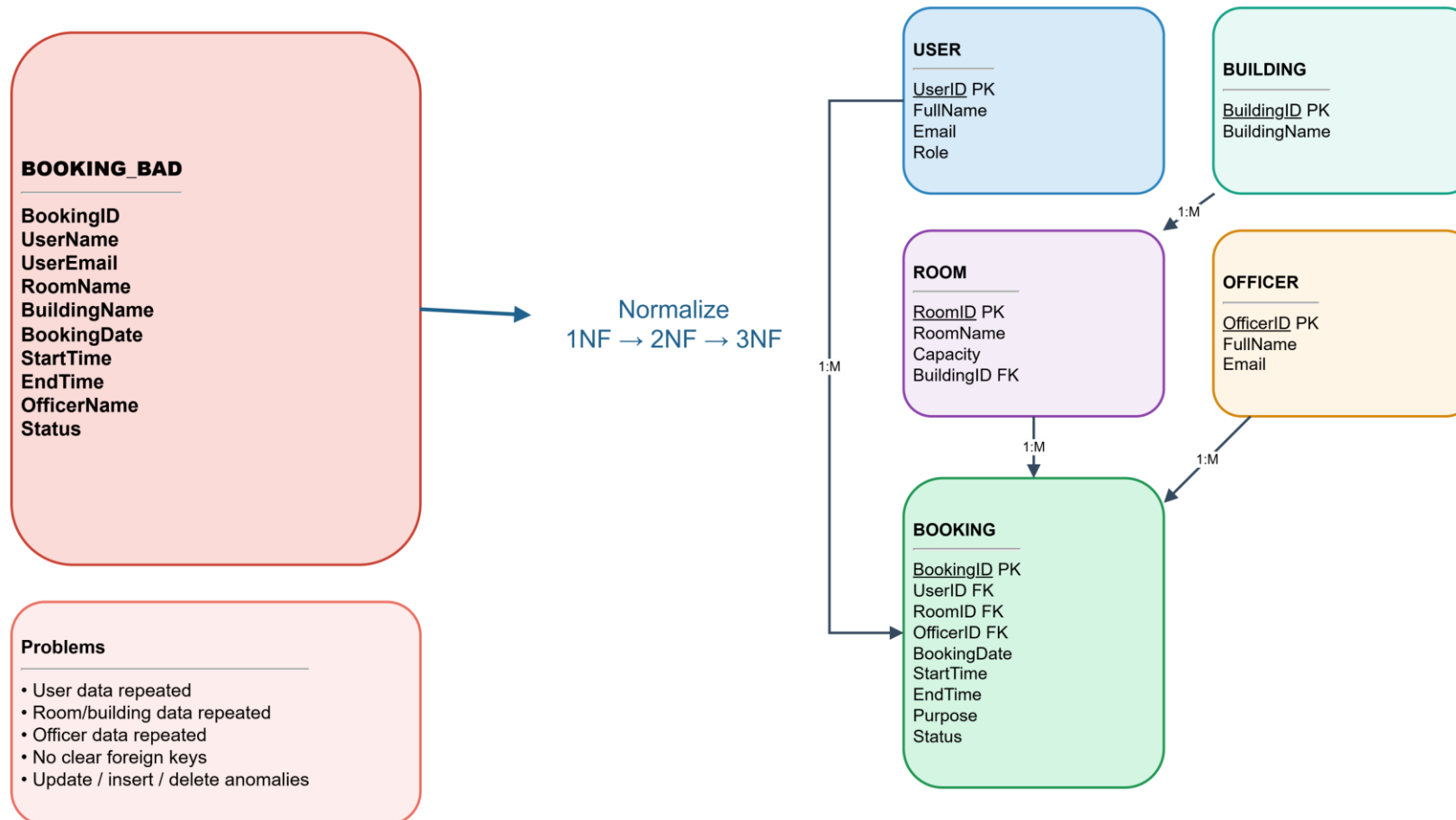
- Reduce unnecessary duplication.
- Improve consistency.
- Avoid update anomalies.
- Avoid insert anomalies.
- Avoid delete anomalies.
- Create relations that are easier to maintain.

Common normal forms:

- 1NF: Atomic values, no repeating groups.
- 2NF: No partial dependency on part of a composite key.
- 3NF: No transitive dependency between non-key attributes.

Normalization: Before and After

Before: repeated data and anomalies. After: separated tables with keys



First Normal Form (1NF)

A table is in 1NF when:

- Each cell contains a single value.
- There are no repeating groups.
- Each row can be uniquely identified.

Bad example

Student(StudentID, FullName, Phone1, Phone2, Phone3)

Improved design

STUDENT(StudentID PK, FullName)

STUDENT_PHONE(StudentID FK, PhoneNumber)

Second Normal Form (2NF)

A table is in 2NF when:

- It is already in 1NF.
- Every non-key attribute depends on the whole primary key, not only part of a composite key.

Problem example

ENROLLMENT(StudentID, CourseID, StudentName, CourseName, EnrollDate)

Issue

- StudentName depends only on StudentID.
- CourseName depends only on CourseID.
- Composite PK = StudentID + CourseID

Third Normal Form (3NF)

A table is in 3NF when:

- It is already in 2NF.
- Non-key attributes do not depend on other non-key attributes.

Problem example

STUDENT(StudentID, FullName, DepartmentID, DepartmentName)

Issue

DepartmentName depends on DepartmentID, not directly on StudentID.

Improved design

STUDENT(StudentID PK, FullName, DepartmentID FK)

DEPARTMENT(DepartmentID PK, DepartmentName).

Common Data Anomalies

Update anomaly

- The same data must be updated in many rows.

Insert anomaly

- A new fact cannot be inserted unless another unrelated fact exists.

Delete anomaly

- Deleting one record accidentally removes important information.

Example

If room information is stored only inside bookings, deleting the last booking may also remove the only record of that room

Example: Poor Database Table

BOOKING_BAD(BookingID, UserName, UserEmail, RoomName, BuildingName, BookingDate, StartTime, EndTime, OfficerName, Status)

Problems

- User data is repeated across bookings.
- Room and building data are repeated.
- Officer information is repeated.
- No clear foreign keys.
- Difficult to enforce valid room and officer records.

Example: Improved Database Schema

Improved schema:

- SYSTEM_USER(UserID PK, FullName, Email, Role)
- BUILDING(BuildingID PK, BuildingName)
- ROOM(RoomID PK, RoomName, Capacity, BuildingID FK)
- OFFICER(OfficerID PK, FullName, Email)
- BOOKING(BookingID PK, UserID FK, RoomID FK, OfficerID FK, BookingDate, StartTime, EndTime, Purpose, Status)

Benefits

- Less duplication.
- Clear relationships.
- Easier validation.
- Better reporting.

Data Dictionary Example

Field	Type	Constraint	Description	Required
BookingID	CHAR(12)	PK	Unique booking number	Yes
RoomID	CHAR(8)	FK	Selected room	Yes
Status	VARCHAR(20)	Submitted/ Approved/ Rejected	Booking state	Yes

Constraints in Database Design

Application validation prevents bad input early; database constraints protect stored data even if bad data bypasses the application

Common constraints:

- PRIMARY KEY
- FOREIGN KEY
- NOT NULL
- UNIQUE
- CHECK
- DEFAULT

```
CREATE TABLE document_request (  
    request_id INT PRIMARY KEY,  
    student_id INT NOT NULL,  
    type_id INT NOT NULL,  
    request_date DATE NOT NULL,  
    purpose VARCHAR(255),  
    status VARCHAR(30) DEFAULT 'Submitted',  
    CONSTRAINT fk_request_student  
        FOREIGN KEY (student_id) REFERENCES student(student_id),  
    CONSTRAINT fk_request_type  
        FOREIGN KEY (type_id) REFERENCES document_type(type_id),  
    CONSTRAINT chk_request_status  
        CHECK (status IN ('Draft', 'Submitted', 'Under Review', 'Approved', 'Rejected', 'Revision Required'))  
);
```

Physical Field Design

Physical field design defines:

- Field name.
- Data type.
- Field length.
- Required or optional status.
- Default value.
- Format or template.
- Range control.
- Null value control.

Example

Status VARCHAR(20) NOT NULL DEFAULT 'Submitted'
Capacity INT CHECK (Capacity > 0)

Data Integrity Controls

Data integrity means stored data remains accurate, consistent, and trustworthy.

Common controls

- NOT NULL for required fields.
- UNIQUE for values that cannot be duplicated.
- CHECK for valid ranges or categories.
- FOREIGN KEY for valid relationships.

Default values for standard initial states.

Indexes and Performance

- An index helps the database find rows faster.
- Indexes are useful for frequently searched, filtered, joined, or sorted columns.
- Indexes improve retrieval but may slow insert, update, and delete operations.

Good index candidates

- Primary keys.
- Foreign keys.
- Email.
- BookingDate.
- Status

Normalization vs Performance

- Normalization improves consistency.
- Too many joins may affect performance.
- Denormalization may be used carefully in physical design.
- Decision should be based on reporting and performance need.

Example:

Monthly summary report may use a summary table for performance.

Database Design and Reports

Reports depend on database design.

Example report:

Monthly Room Utilization

Needed data:

- RoomID and RoomName.
- BookingDate.
- StartTime and EndTime.
- BookingStatus.
- BuildingName.

Design implication

The database must store time, status, and room-building relationships accurately.

Database Design Checklist

Use this checklist before finalizing a database design:

Does every table have a clear purpose?

Does every table have a primary key?

Are foreign keys placed correctly?

Are M:N relationships resolved?

Are repeated groups removed?

Are business rules represented as constraints?

Are important searches supported by indexes?

Can required reports be produced from the tables?

Practice 2: Database Schema Design

Scenario: University Facility Booking System

A university system allows students and staff to book rooms. Each room belongs to one building. A booking is submitted by one user and reviewed by one facility officer. A booking has booking date, start time, end time, purpose, number of participants, and status. A booking may request multiple equipment items such as projector, microphone, or speaker.

Tasks

1. Identify main tables.
2. Identify primary keys and foreign keys.
3. Resolve any many-to-many relationship.
4. Add at least five constraints.
5. Identify two fields that should be indexed.

Summary: Database Design

Database design transforms conceptual data models into implementable database structures.

Remember:

- Entity becomes table.
- Attribute becomes column.
- Identifier becomes primary key.
- 1:N relationship is implemented using a foreign key.
- M:N relationship requires an associative table.
- Normalization helps reduce redundancy and update anomalies.
- Constraints protect data integrity.
- SQL schema is the technical output of database design.

Comparison: LLD and Database Design

Aspect	Low-Level Design	Database Design
Focus	Behavior and interaction	Data structure and integrity
Main model	Sequence diagram, interface contract	ERD, relational schema
Output	Component specification	SQL schema
Used by	Developers, testers	Developers, DB designers

Practice: Database Design

Scenario: Student Academic Document Request System

A student submits a request for an academic document, such as a transcript or enrollment certificate. The student fills in a request form, uploads supporting documents, and submits the request. The system validates the request information. If the request is valid, the Request Module saves the request and asks the Notification Module to send a confirmation to the student. Later, an academic affairs staff member reviews the request. If the request is incomplete, the staff member marks it as “Revision Required,” and the system sends a notification to the student. The student revises the request and uploads the missing document again.

Practice: Database Design

Database Design

Convert this mini ERD into schema:

- Student submits Request.
- DocumentRequest is for one DocumentType.
- DocumentRequest has UploadedDocument.
- Staff reviews DocumentRequest.
- DocumentRequest generates Notification.
- Student receives Notification.

Deliverables: 1 relational schema list ; Identify PK and FK clearly